

Специфікація вимог до програмного забезпечення (SRS)

«Експрес Львів–Дрогобич» — Автоматизована омніканальна система бронювання

Telegram Bot + Web Admin Panel

Версія документа: 1.0
Дата: 27 лютого 2026
Статус: Draft — передано замовнику на узгодження

Історія змін документа

Версія	Дата	Автор	Опис змін
1.0	27.02.2026	Команда аналітиків	Початкова версія документа

Зміст

- [Вступ](#)
 - [1.1 Призначення](#)
 - [1.2 Область застосування](#)
 - [1.3 Визначення, акроніми та аббревіатури](#)
 - [1.4 Посилання](#)
 - [1.5 Огляд документа](#)
- [Загальний опис](#)
 - [2.1 Перспектива продукту](#)
 - [2.2 Функції продукту](#)
 - [2.3 Характеристики користувачів](#)
 - [2.4 Обмеження](#)
 - [2.5 Припущення та залежності](#)
- [Специфічні вимоги](#)
 - [3.1 Функціональні вимоги](#)
 - [3.2 Вимоги до інтерфейсу користувача](#)
 - [3.3 Нефункціональні вимоги](#)
- [Додатки](#)

1. Вступ

1.1 Призначення

Даний документ є специфікацією вимог до програмного забезпечення (SRS) для автоматизованої омніканальної системи бронювання «Експрес Львів–Дрогобич». Документ призначений для:

- Команди розробників, які здійснюватимуть реалізацію системи
- Архітекторів програмного забезпечення, відповідальних за проектування архітектури
- Тестувальників, які здійснюватимуть перевірку відповідності реалізації вимогам
- Менеджерів проєкту для планування робіт
- Замовників та стейкхолдерів для узгодження функціональності системи

Документ визначає функціональні та нефункціональні вимоги до системи, описує взаємодію між компонентами та визначає критерії прийняття готового продукту.

1.2 Область застосування

Назва системи: «Експрес Львів–Дрогобич»

Опис системи: Автоматизована омніканальна платформа, що замінює ручний облік пасажирів (паперові блокноти та телефонні дзвінки) для міжміських мікроавтобусних маршрутів. Ключова особливість — єдина централізована база даних (PostgreSQL) для замовлень з усіх каналів взаємодії: самостійне бронювання пасажиром через Telegram Bot, бронювання диспетчером за телефонним дзвінком, та фіксація посадки водієм на маршруті.

Цілі системи:

- Замінити ручний облік централізованою базою даних

- Надати пасажирам самостійний доступ до бронювання через Telegram Bot
- Забезпечити водіям швидкий інструмент для управління посадкою без ручного введення даних
- Надати диспетчерам і власнику Web-панель для управління розкладом, транспортом та аналітикою
- Виключити можливість overbooking завдяки транзакційним блокуванням на рівні БД

Межі системи:

Система охоплює:

- Самостійне бронювання пасажирями через Telegram Bot
- Ручне бронювання диспетчером за телефонним дзвінком
- Управління посадкою водієм (стоячі пасажирі, посилки, boarding validation)
- Генерацію та управління розкладом рейсів
- Управління автопарком та конфігурацією транспортних засобів
- Фінансову аналітику, аудит-слід та Trust Score для власника
- Систему ролей на основі булевих прапорців

Система **не** охоплює:

- Онлайн-платежі та еквайринг
- Юридичне оформлення транспортних документів
- Інтеграцію з державними реєстрами
- Мобільний застосунок (iOS / Android)
- Маркетплейс або рекламний модуль

Вигоди від впровадження:

- Виключення людського фактору при обліку місць
- Централізований журнал усіх бронювань з усіх каналів
- Можливість аналізувати завантаженість рейсів та надійність пасажирів
- Зменшення навантаження на диспетчерів
- Повний аудит-слід: хто, коли і звідки вніс кожне бронювання

1.3 Визначення, акроніми та аббревіатури

Термін / Аббревіатура	Визначення
SRS	Software Requirements Specification — Специфікація вимог до програмного забезпечення
API	Application Programming Interface — Програмний інтерфейс застосунку
БД	База даних
ORM	Object-Relational Mapping — Об'єктно-реляційне відображення
JWT	JSON Web Token — токен автентифікації
ACID	Atomicity, Consistency, Isolation, Durability — Властивості транзакцій БД
Race Condition	Стан системи, при якому два або більше конкурентних запити одночасно претендують на одне місце
Snapshot	Незмінна копія значення на момент створення запису (не оновлюється при зміні джерела)
Тіньовий профіль	Запис у таблиці USERS з telegram_id = NULL , створений диспетчером для клієнта без Telegram
Trust Score	Кешований рейтинг надійності пасажиря (0–100), знижується за кожну неявку
No-Show	Пасажир, що забронював місце, але не з'явився на посадку до відправлення рейсу
Boarding Validation	Підтвердження водієм факту фізичної посадки пасажиря: заповнення validated_by_id та validated_at
Маніфест рейсу	Список усіх активних бронювань рейсу, що використовується водієм для управління посадкою
Overbooking	Ситуація, де кількість бронювань перевищує фізичну місткість транспортного засобу
CLOSED	Фінальний незворотний статус рейсу після фінансової звірки диспетчера з водієм
Source	Канал, через який надійшло бронювання: BOT / WEB / PHONE / INSTAGRAM / DRIVER
Audit Trail	Повний журнал дій через поля created_by_id і validated_by_id у таблиці BOOKINGS

1.4 Посилання

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications
- ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering
- FastAPI Documentation — <https://fastapi.tiangolo.com>
- SQLAlchemy 2.0 Documentation — <https://docs.sqlalchemy.org/en/20/>
- aiogram 3.x Documentation — <https://docs.aiogram.dev/en/latest/>
- PostgreSQL 15 Documentation — <https://www.postgresql.org/docs/15/>
- Alembic Documentation — <https://alembic.sqlalchemy.org/en/latest/>

1.5 Огляд документа

Розділ 2 надає загальний опис системи «Експрес Львів–Дрогобич», включаючи архітектуру підсистем, ролі акторів, основні функції, обмеження та припущення щодо функціонування.

Розділ 3 містить детальний опис функціональних вимог для кожної підсистеми (Telegram Bot — Пасажир, Telegram Bot — Водій, Web Admin Panel), вимог до інтерфейсу користувача, а також нефункціональних вимог до продуктивності, безпеки, надійності та технологічного стеку.

Розділ 4 містить додаткові матеріали: Use Case діаграми, схему бази даних, MoSCoW пріоритетизацію, ризики проекту та глосарій.

2. Загальний опис

2.1 Перспектива продукту

Система «Експрес Львів–Дрогобич» є самостійним програмним продуктом, що функціонує за архітектурою omni-channel з єдиним Core API. Система складається з наступних основних компонентів:

Telegram Bot UI (Client & Driver Layer):

- Розробляється з використанням Python та бібліотеки aiogram 3.x
- Два режими роботи: пасажирський (для всіх зареєстрованих) та водійський (для `is_driver = true`)
- Інтерфейс на основі inline-клавіатур без ручного введення тексту для водія
- Не містить бізнес-логіки — виключно UI-прошарок поверх Core API

Web Admin Panel (Management Layer):

- Веб-інтерфейс для диспетчерів та власника
- Взаємодія з Core API через REST-запити
- Забезпечує візуалізацію розкладу, CRM для пасажирів, управління автопарком

Core API & Booking Engine (Service Layer):

- Розробляється на базі FastAPI (Python)
- Центральний сервіс бізнес-логіки: валідація лімітів, транзакційні блокування, генерація розкладу
- Захист від Race Condition через `SELECT ... FOR UPDATE` в транзакціях PostgreSQL
- Повний аудит-слід усіх операцій

База даних:

- PostgreSQL 15 як основна СУБД
- Взаємодія через SQLAlchemy 2.0 з асинхронним драйвером `asynccpg`
- Версіонування схеми через Alembic

2.2 Функції продукту

Управління бронюваннями:

- Самостійне бронювання сидячих місць пасажиром через бот
- Ручне бронювання диспетчером за телефонним дзвінком
- Швидка фіксація стоячих пасажирів та посилок водієм в 1 клік
- Скасування бронювань пасажиром та диспетчером

Управління рейсами:

- Генерація розкладу на день/тиждень
- Зміна статусу рейсу (SCHEDULED → BOARDING → ACTIVE → COMPLETED → CLOSED)
- Фінансове закриття рейсу після звірки каси

Управління автопарком:

- Додавання та редагування транспортних засобів
- Snapshot-копіювання лімітів місць при створенні рейсу

Аналітика та безпека:

- Trust Score та статистика надійності пасажирів
- Блокування ненадійних пасажирів
- Повний аудит-слід: хто і коли створив або підтвердив кожне бронювання
- Маркування джерела бронювання (BOT / WEB / PHONE / INSTAGRAM / DRIVER)

2.3 Характеристики користувачів

Важливо: Ролі реалізовані через булеві прапорці (`is_driver`, `is_admin`, `is_superuser`) у таблиці `USERS`. Одна фізична особа може поєднувати кілька ролей одночасно.

2.3.1 Пасажир (Passenger)

Технічна експертиза: Базовий рівень — вміє користуватися Telegram

Основні цілі: Знайти зручний рейс та забронювати місце самостійно без дзвінка диспетчеру

Ідентифікація: Telegram ID + номер телефону (через шеринг контакту)

Прапорці: `is_driver = false`, `is_admin = false`, `is_superuser = false`

Функції:

- Реєстрація через шеринг контакту Telegram
- Пошук рейсів на обрану дату та напрямок
- Бронювання сидячого місця з перевіркою наявності
- Перегляд активних квитків та історії поїздок
- Скасування власного бронювання

Обмеження:

- Не має доступу до даних інших пасажирів
- Не може змінювати статус рейсу
- Заблокований пасажир (`is_active = false`) не може бронювати

2.3.2 Водій (Driver)

Технічна експертиза: Базовий рівень — звик до швидких дій під час руху

Основні цілі: Швидко керувати посадкою без відволікання від дороги

Прапорці: `is_driver = true`

Функції:

- Перегляд маніфесту свого поточного рейсу
- Зміна статусу рейсу (`BOARDING` → `ACTIVE` → `COMPLETED`)
- Додавання стоячого пасажирів в 1 клік (без введення імені/телефону)
- Запис посилки з опціональним коментарем
- Підтвердження посадки заброньованого пасажирів

Обмеження:

- Не вводить імена чи номери телефонів під час руху
- Не має доступу до адмін-панелі
- Не може перевести рейс у `CLOSED` або `CANCELLED`

2.3.3 Диспетчер (Admin)

Технічна експертиза: Середній рівень — досвід роботи з веб-формами

Основні цілі: Приймати замовлення по телефону та вести облік рейсів

Прапорці: `is_admin = true`

Функції:

- Генерація розкладу на день/тиждень
- Ручне створення бронювань за телефонними дзвінками клієнтів
- Скасування рейсів та бронювань
- Вирішення конфліктних ситуацій
- Фінансове закриття рейсів (статус `CLOSED`)

Характеристики:

- Кожне ручне бронювання фіксується з `created_by_id = dispatcher.id` (аудит)
- Може створювати тінюві профілі для клієнтів без Telegram

2.3.4 Власник / Супер-Адмін (Owner)

Технічна експертиза: Середній-високий рівень

Основні цілі: Контролювати бізнес, бачити «касу», управляти персоналом і автопарком

Прапорці: `is_superuser = true`

Функції:

- Повний доступ до всього функціоналу диспетчера
- Перегляд фінансової статистики та `audit trails`
- Управління автопарком (додавання та редагування `VEHICLES`)
- Призначення ролей співробітникам (`is_driver`, `is_admin`)
- Перегляд `Trust Score` пасажирів та управління блокуваннями

2.4 Обмеження

2.4.1 Технологічні обмеження

- Backend розробляється виключно на Python (FastAPI)
- База даних: PostgreSQL 15+; заміна на інші СУБД не передбачена
- Telegram Bot: виключно aiogram 3.x з асинх-підходом
- ORM: SQLAlchemy 2.0 + asynpcrg; синхронний доступ до БД заборонений

2.4.2 Бізнес-обмеження

- Один рейс може мати лише одного водія та один транспортний засіб
- Посилки (PARCEL) не займають місць у салоні (`passengers_count = 0`)
- Поле `created_by_id` у BOOKINGS обов'язкове і не може бути NULL
- Зміна конфігурації авто не впливає на вже створені рейси (Snapshot-принцип)

2.4.3 Нормативні обмеження

- Паролі адміністраторів зберігаються лише у хешованому вигляді (bcrypt або argon2)
- Персональні дані пасажирів обробляються відповідно до законодавства України
- Доступ до фінансових даних обмежений виключно для `is_superuser = true`

2.4.4 Організаційні обмеження

- Розробка ведеться у рамках визначеного бюджету та часових рамок MVP
- Усі зовнішні зміни вимог фіксуються у вигляді оновлень цього документа

2.5 Припущення та залежності

2.5.1 Припущення

- Пасажири мають встановлений Telegram на смартфоні
- Водії мають смартфон з доступом до Telegram під час рейсів
- Диспетчер має постійний доступ до веб-браузера на ПК
- Наявне стабільне інтернет-з'єднання на всіх точках використання
- Замовник надає актуальний розклад рейсів та дані автопарку для початкового наповнення

2.5.2 Залежності

Технологічні залежності:

- Доступність серверів PostgreSQL
- Доступність Telegram Bot API (api.telegram.org)
- Стабільність asynpcrg та SQLAlchemy 2.0 для асинх-операцій

Зовнішні залежності:

- Telegram Bot API — для відправки повідомлень та отримання webhook-подій
- SMTP-сервіс або аналог — для адмін-нотифікацій (опціонально)

Правові залежності:

- Дотримання умов використання Telegram Bot API
- Відповідність вимогам щодо збереження персональних даних (телефони пасажирів)

3. Специфічні вимоги

3.1 Функціональні вимоги

3.1.1 Модуль Telegram Bot — Пасажир

UC-P1: Реєстрація (шеринг контакту Telegram)

Пріоритет: Критичний (Must Have)

Залежності: Відсутні

Опис: Система повинна ідентифікувати пасажирів через стандартний механізм шерингу контакту Telegram та зберегти його профіль у базі даних.

Вхідні дані:

- Telegram ID (отримується автоматично)
- Номер телефону (через шеринг контакту Telegram)

Елементи інтерфейсу бота:

- Привітальне повідомлення при /start
- Кнопка «Поділитися номером телефону» (Telegram request_contact)
- Повідомлення про успішну реєстрацію
- Головне меню пасажирів

Функціональність:

- UC-P1.1: Відображення кнопки шерингу контакту при першому запуску або спробі бронювання без реєстрації
- UC-P1.2: Отримання telegram_id та phone від Telegram API після підтвердження
- UC-P1.3: Перевірка наявності запису у USERS з таким phone або telegram_id
- UC-P1.4: Якщо запис відсутній — Core API створює новий USERS та початковий USER_STATS (trust_score_cached = 100 , total_trips = 0 , total_noshows = 0)
- UC-P1.5: Якщо існує тінювий профіль (створений диспетчером) — Core API оновлює запис, прив'язуючи telegram_id
- UC-P1.6: Перехід до головного меню пасажирів після успішної реєстрації

Критерії прийняття:

- Реєстрація завершується менш ніж за 3 секунди
- Тінювий профіль коректно об'єднується без дублювання записів
- Повторна реєстрація з тим самим телефоном не створює нового запису

UC-P2: Пошук рейсів на обрану дату

Пріоритет: Критичний (Must Have)

Залежності: UC-P1

Опис: Система повинна дозволяти пасажирів знаходити доступні рейси за обраним напрямком та датою.

Елементи інтерфейсу бота:

- Кнопки вибору напрямку: «Львів → Дрогобич» / «Дрогобич → Львів»
- Inline-клавіатура вибору дати: «Сьогодні», «Завтра», та дати на 5 днів вперед
- Список карток рейсів: час відправлення, ціна, кількість вільних місць
- Кнопка «Назад» до головного меню

Функціональність:

- UC-P2.1: Відображення вибору напрямку після натискання «Знайти рейс»
- UC-P2.2: Відображення inline-клавіатури вибору дати після вибору напрямку
- UC-P2.3: Запит до Core API: фільтрація TRIPS за from_location_id, to_location_id, датою departure_time, статусом SCHEDULED або BOARDING
- UC-P2.4: Для кожного рейсу обчислення seats_available = seats_limit_snapshot - SUM(passengers_count WHERE booking_type='SEATED' AND status IN ('RESERVED', 'PAID', 'BOARDED'))
- UC-P2.5: Відображення рейсів з кількістю вільних місць; рейси без місць позначені «Місць немає» та недоступні для вибору
- UC-P2.6: Перехід до UC-P3 після вибору рейсу

Критерії прийняття:

- Список рейсів відображається менш ніж за 2 секунди
- Рейси без місць не дозволяють перейти до бронювання
- На рейси зі статусом CANCELLED або COMPLETED немає результатів

UC-P3: Бронювання сидячого місця

Пріоритет: Критичний (Must Have)

Залежності: UC-P2

Опис: Система повинна дозволяти пасажирів забронювати сидяче місце на обраний рейс із захистом від overbooking при конкурентних запитах.

Передумова: Рейс має статус SCHEDULED або BOARDING; є принаймні 1 вільне сидяче місце за даними попереднього екрану

Елементи інтерфейсу бота:

- Картка підтвердження: маршрут, дата, час відправлення, ціна
- Кнопка «Забронювати»
- Кнопка «Назад»
- Повідомлення про успішне бронювання з деталями квитка
- Повідомлення про помилку (Race Condition або блокування)

Функціональність:

- UC-P3.1: Відображення деталей рейсу та кнопки підтвердження
- UC-P3.2: Після натискання «Забронювати» — Core API відкриває транзакцію БД
- UC-P3.3: Core API виконує SELECT seats_limit_snapshot FROM trips WHERE id = :trip_id FOR UPDATE — встановлює рядкове блокування
- UC-P3.4: Перевірка актуальної кількості вільних місць у межах транзакції: seats_available = seats_limit_snapshot - SUM(passengers_count WHERE booking_type='SEATED' AND status IN ('RESERVED', 'PAID', 'BOARDED'))
- UC-P3.5: Якщо seats_available ≥ 1 — Core API створює запис BOOKINGS: booking_type = 'SEATED', source = 'BOT', status = 'RESERVED', created_by_id = passenger.id, passenger_id = passenger.id, passengers_count = 1
- UC-P3.6: Commit транзакції; бот відправляє підтвердження з деталями квитка
- UC-P3.7: Якщо seats_available = 0 після отримання блокування (Race Condition) — Rollback; бот повідомляє пасажирів

Критерії прийняття:

- При одночасному бронюванні останнього місця двома пасажирами — лише один отримує підтвердження
- Час відповіді бота: ≤ 3 секунди
- Заблокований пасажир (`is_active = false`) отримує повідомлення про неможливість бронювання

UC-P4: Перегляд «Моїх квитків»

Пріоритет: Критичний (Must Have)

Залежності: UC-P3

Опис: Система повинна надавати пасажиру доступ до його активних бронювань та історії поїздок.

Елементи інтерфейсу бота:

- Список активних квитків (RESERVED, PAID, BOARDED; `departure_time` у майбутньому)
- Список минулих поїздок (BOARDED зі старою датою, CANCELLED, NOSHOW)
- Картка квитка: маршрут, дата, час відправлення, статус
- Кнопка «Скасувати» для активних квитків
- Кнопка «Моя історія» для перегляду минулих поїздок
- Повідомлення про відсутність квитків

Функціональність:

- UC-P4.1: Core API отримує всі BOOKINGS де `passenger_id = current_user.id`
- UC-P4.2: Розподіл на дві групи: **Активні** — статус RESERVED / PAID / BOARDED та `departure_time` у майбутньому; **Історія** — решта
- UC-P4.3: Відображення активних квитків з кнопкою «Скасувати» для кожного (якщо застосовно)
- UC-P4.4: Перехід до UC-P5 після натискання «Скасувати»
- UC-P4.5: Відображення порожнього стану з кнопкою «Знайти рейс», якщо бронювань немає

Критерії прийняття:

- Список завантажується менш ніж за 2 секунди
- Квитки коректно розподіляються на активні та архівні
- Для вже відправлених рейсів кнопка «Скасувати» не відображається

UC-P5: Скасування свого бронювання

Пріоритет: Критичний (Must Have)

Залежності: UC-P4

Опис: Система повинна дозволяти пасажиру скасувати власне бронювання до відправлення рейсу.

Передумова: Бронювання має статус RESERVED або PAID; `departure_time` рейсу у майбутньому

Елементи інтерфейсу бота:

- Запит підтвердження: «Скасувати бронювання на [маршрут, дата, час]?»
- Кнопки «Так, скасувати» та «Ні, повернутись»
- Підтвердження успішного скасування
- Повідомлення про неможливість скасування (якщо рейс вже відправився)

Функціональність:

- UC-P5.1: Відображення запиту підтвердження з деталями бронювання
- UC-P5.2: Core API перевіряє: статус бронювання RESERVED / PAID та `departure_time` рейсу у майбутньому
- UC-P5.3: Core API змінює status запису BOOKINGS на CANCELLED
- UC-P5.4: Бот відправляє підтвердження; місце стає вільним для інших пасажирів
- UC-P5.5: Якщо рейс вже відправився (статус ACTIVE / COMPLETED) або `departure_time` у минулому — Core API повертає помилку; бот відображає пояснення
- UC-P5.6: Якщо пасажир натискає «Ні» — бот повертається до «Мої квитки» без змін

Критерії прийняття:

- Скасування застосовується миттєво
- Після скасування місце стає доступним для нового бронювання
- Неможливо скасувати бронювання рейсу, що вже відправився

3.1.2 Модуль Telegram Bot — Водій

UC-D1: Перегляд маніфесту поточного рейсу

Пріоритет: Критичний (Must Have)

Залежності: Призначення водія на рейс (UC-A1)

Опис: Система повинна надавати водієві повний список пасажирів та бронювань для управління посадкою.

Передумова: Є рейс де `driver_id = current_user.id` зі статусом SCHEDULED, BOARDING або ACTIVE

Елементи інтерфейсу бота:

- Список призначених рейсів з датою та часом відправлення
- Маніфест: список SEATED-пасажирів (ім'я, телефон), кількість STANDING, кількість PARCEL
- Загальна статистика рейсу: всього пасажирів, завантаженість
- Кнопки дій для кожного пасажирів: «✓ Підтвердити посадку»
- Кнопки швидких дій: «+ Стоячий», «📄 Посилка», зміна статусу рейсу

Функціональність:

- UC-D1.1: Відображення списку рейсів водія при відкритті меню «Мої рейси»
- UC-D1.2: Core API повертає всі BOOKINGS для обраного trip_id зі статусом, відмінним від CANCELLED
- UC-D1.3: Відображення маніфесту з групуванням: SEATED (з іменем/телефоном), STANDING (кількість), PARCEL (кількість)
- UC-D1.4: Для пасажирів зі статусом RESERVED / PAID — кнопка підтвердження посадки (UC-D5)
- UC-D1.5: Для пасажирів зі статусом BOARDED — позначка «✓ На борту»

Критерії прийняття:

- Маніфест (до 50 записів) завантажується менш ніж за 3 секунди
- Відображається актуальна кількість пасажирів після кожної зміни
- При відсутності призначених рейсів — «На сьогодні рейсів не призначено»

UC-D2: Зміна статусу рейсу

Пріоритет: Критичний (Must Have)

Залежності: UC-D1

Опис: Система повинна дозволяти водієві змінювати статус рейсу відповідно до допустимих переходів.

Допустимі переходи: SCHEDULED → BOARDING → ACTIVE → COMPLETED

Елементи інтерфейсу бота:

- Поточний статус рейсу у заголовку маніфесту
- Кнопка наступного статусу: «Розпочати посадку» / «Вирушити» / «Завершити рейс»
- Запит підтвердження для «Завершити рейс»
- Повідомлення про неможливість переходу

Функціональність:

- UC-D2.1: Відображення кнопки наступного статусу відповідно до поточного
- UC-D2.2: Core API валідує допустимість переходу та оновлює TRIPS.status
- UC-D2.3: Бот підтверджує зміну та оновлює меню
- UC-D2.4: Зворотні або неприпустимі переходи відхиляються з поясненням

Критерії прийняття:

- Зміна статусу виконується менш ніж за 2 секунди
- Заборонені переходи відхиляються API з помилкою 400
- Водій не може перевести рейс у CANCELLED або CLOSED

UC-D3: Швидкий продаж «Стоячого» місця

Пріоритет: Критичний (Must Have)

Залежності: UC-D1

Опис: Система повинна дозволяти водієві зафіксувати стоячого пасажирів в 1 клік без введення особистих даних.

Передумова: Рейс має статус BOARDING або ACTIVE ; є вільні стоячі місця

Елементи інтерфейсу бота:

- Кнопка «+ Стоячий» у меню рейсу
- Миттєве підтвердження: «Стоячий додано. Стоячих: X / Y»
- Повідомлення про вичерпаний ліміт

Функціональність:

- UC-D3.1: Core API перевіряє: standing_available = standing_limit_snapshot - SUM(passengers_count WHERE booking_type='STANDING' AND status IN ('RESERVED','PAID','BOARDED'))
- UC-D3.2: Якщо standing_available ≥ 1 : Core API створює BOOKINGS: booking_type = 'STANDING', source = 'DRIVER', status = 'BOARDED', passenger_id = NULL, created_by_id = driver.id, passengers_count = 1
- UC-D3.3: Бот відображає підтвердження з оновленим лічильником
- UC-D3.4: Якщо standing_available = 0 — повідомлення «Ліміт стоячих вичерпано»

Критерії прийняття:

- Операція виконується в 1 клік без введення будь-якого тексту
- Час реакції бота: ≤ 2 секунди
- Лічильник стоячих оновлюється миттєво

UC-D4: Швидкий запис «Посилки»

Пріоритет: Критичний (Must Have)

Залежності: UC-D1

Опис: Система повинна дозволяти водієві зафіксувати прийняту посилку, яка не займає місця в салоні.

Елементи інтерфейсу бота:

- Кнопка «📄 Посилка» у меню рейсу
- Опціональний запит коментаря (з кнопкою «Пропустити»)
- Підтвердження: «Посилку записано»

Функціональність:

- UC-D4.1: Натискання кнопки «Посилка» відкриває опціональний ввід коментаря
- UC-D4.2: Коментар необов'язковий — кнопка «Пропустити» одразу створює запис
- UC-D4.3: Core API створює `BOOKINGS`: `booking_type = 'PARCEL'`, `passengers_count = 0`, `source = 'DRIVER'`, `status = 'BOARDED'`, `passenger_id = NULL`, `created_by_id = driver.id`
- UC-D4.4: Посилка не враховується в жодному ліміті місць

Критерії прийняття:

- Запис посилки без коментаря виконується в 1–2 кліки
- `passengers_count = 0` завжди для PARCEL
- Кількість посилки відображається в маніфесті окремою групою

UC-D5: Підтвердження посадки (Boarding Validation)

Пріоритет: Критичний (Must Have)

Залежності: UC-D1

Опис: Система повинна дозволяти водієві відмітити факт фізичної посадки пасажирів, що має попереднє бронювання.

Передумова: Рейс у статусі `BOARDING`; у маніфесті є пасажир зі статусом `RESERVED` або `PAID`

Елементи інтерфейсу бота:

- У маніфесті поруч з кожним незаборденим пасажиром — кнопка «✓ Підтвердити»
- Після підтвердження — пасажир позначається «✓ На борту» в маніфесті

Функціональність:

- UC-D5.1: Core API оновлює запис `BOOKINGS`: `status = 'BOARDED'`, `validated_by_id = driver.id`, `validated_at = NOW()`
- UC-D5.2: Маніфест оновлюється — пасажир переміщується у групу «На борту»

No-Show сценарій (автоматичний або диспетчерський):

- UC-D5.3: Після переходу рейсу в `ACTIVE` або `COMPLETED` — всі бронювання зі статусом `RESERVED` / `PAID` автоматично переводяться в `NOSHOW`
- UC-D5.4: Core API збільшує `USER_STATS.total_noshows` та перераховує `trust_score_cached` для відповідних пасажирів

Критерії прийняття:

- Підтвердження посадки виконується в 1 клік
- `validated_by_id` та `validated_at` заповнюються коректно
- Пасажири, що не підтвердили посадку, автоматично отримують статус `NOSHOW`

3.1.3 Модуль Web Admin Panel — Диспетчер / Власник

UC-A1: Генерація розкладу на день / тиждень

Пріоритет: Критичний (Must Have)

Залежності: UC-A5 (наявність VEHICLES)

Опис: Система повинна дозволяти адміністратору швидко створювати розклад рейсів на вибраний період.

Пріоритет: Критичний (Must Have)

Елементи інтерфейсу Web-панелі:

- Форма вибору діапазону дат (один день або тиждень)
- Шаблон розкладу з полями: час відправлення, маршрут, транспортний засіб, водій
- Можливість редагування перед збереженням
- Список щойно створених рейсів із підтвердженням

Функціональність:

- UC-A1.1: Адміністратор обирає діапазон дат та шаблон розкладу
- UC-A1.2: Для кожного рейсу у шаблоні Core API: створює запис `TRIPS`, копіює `total_seats` та `total_standing` з обраного `VEHICLES` у `seats_limit_snapshot` та `standing_limit_snapshot`, встановлює `status = 'SCHEDULED'`
- UC-A1.3: Система відображає список створених рейсів із підсумком
- UC-A1.4: Core API перевіряє конфлікти: водій або авто вже призначені на цей час в іншому рейсі

Критерії прийняття:

- Генерація тижневого розкладу (14 рейсів) займає менш ніж 5 секунд

- Snapshot-поля заповнюються автоматично з поточної конфігурації VEHICLES
- При конфлікті ресурсів система попереджає до збереження

UC-A2: Ручне створення бронювання (за телефонним дзвінком)

Пріоритет: Критичний (Must Have)

Залежності: UC-A1

Опис: Система повинна дозволяти диспетчеру вносити бронювання від клієнтів, що телефонують, з автоматичним створенням тінювого профілю при необхідності.

Елементи інтерфейсу Web-панелі:

- Форма бронювання: вибір рейсу, поле телефону, поле імені (для нових клієнтів)
- Автопідставлення даних, якщо телефон вже є в базі
- Поле Коментар (опціонально)
- Кнопки «Підтвердити бронювання» та «Скасувати»
- Підтвердження з деталями та `created_by_id`

Функціональність:

- UC-A2.1: Диспетчер обирає рейс та вводить номер телефону клієнта
- UC-A2.2: Core API виконує пошук у `USERS` за `phone`
- UC-A2.3: Якщо знайдено — підтягує ім'я автоматично; якщо ні — диспетчер вводить ім'я і Core API створює тінювий профіль (`telegram_id` = `NULL`)
- UC-A2.4: Core API виконує бронювання з транзакційним захистом: `booking_type` = 'SEATED' , `source` = 'PHONE' , `created_by_id` = `dispatcher.id` , `passenger_id` = `found_or_created_user.id`
- UC-A2.5: Система відображає підтвердження з усіма деталями та аудит-інформацією

Критерії прийняття:

- Бронювання завершується менш ніж за 5 секунд
- `created_by_id` завжди містить ID диспетчера — не пасажирів
- При відсутності місць Core API повертає `SEATS_LIMIT_EXCEEDED` з пропозицією альтернативного рейсу

UC-A3: Фінансове закриття рейсу (статус CLOSED)

Пріоритет: Критичний (Must Have)

Залежності: UC-D2 (рейс у статусі COMPLETED)

Опис: Система повинна дозволяти диспетчеру фінансово закрити завершений рейс після звірки каси з водієм.

Передумова: Рейс має статус `COMPLETED`

Елементи інтерфейсу Web-панелі:

- Картка рейсу з підсумком: кількість `SEATED` / `STANDING` / `PARCEL`, `SUM(amount_paid)` , список усіх бронювань
- Кнопка «Закрити рейс (фінансово)»
- Запит підтвердження (незворотна дія)
- Відображення результату закриття

Функціональність:

- UC-A3.1: Адміністратор переглядає підсумок рейсу та звіряє з водієм
- UC-A3.2: Core API змінює `TRIPS.status` → `CLOSED`
- UC-A3.3: Всі бронювання зі статусом `RESERVED` або `PAID` автоматично переводяться в `NOSHOW`
- UC-A3.4: Core API оновлює `USER_STATS` для відповідних пасажирів
- UC-A3.5: Кнопка «Закрити рейс» недоступна для рейсів з іншим статусом (не `COMPLETED`)

Критерії прийняття:

- Статус `CLOSED` є незворотним — повернення неможливе
- `USER_STATS.total_noshows` оновлюється після закриття
- Фінансовий підсумок відображається коректно до підтвердження

UC-A4: Перегляд Trust Score та блокування пасажирів

Пріоритет: Високий (Should Have)

Залежності: UC-P1 (наявність профілів)

Опис: Система повинна надавати адміністратору інструменти для моніторингу надійності пасажирів та управління блокуваннями.

Елементи інтерфейсу Web-панелі:

- CRM-список пасажирів з колонками: ім'я, телефон, Trust Score, trips / noshows
- Сортування та фільтрація за Trust Score
- Картка пасажира з повною статистикою
- Кнопки «Заблокувати» та «Розблокувати»
- Поле для нотатки про блокування

Функціональність:

- UC-A4.1: Перегляд картки пасажирів: `full_name`, `phone`, `total_trips`, `total_noshows`, `trust_score_cached`
- UC-A4.2: Формула Trust Score: `trust_score = 100 - (total_noshows / MAX(total_trips, 1)) × 100`
- UC-A4.3: Натискання «Заблокувати» → Core API встановлює `USERS.is_active = false`
- UC-A4.4: Заблокований пасажир не може бронювати через бота
- UC-A4.5: Власник може розблокувати пасажирів (`is_active = true`)

Критерії прийняття:

- Trust Score оновлюється після кожного закриття рейсу
- Блокування вступає в силу миттєво
- Спроба бронювання заблокованим пасажиром відхиляється з поясненням

UC-A5: Управління автопарком

Пріоритет: Критичний (Must Have)

Залежності: Відсутні

Опис: Система повинна дозволяти власнику додавати та редагувати транспортні засоби з коректним збереженням snapshot-лімітів при створенні рейсів.

Елементи інтерфейсу Web-панелі:

- Список транспортних засобів: номер, модель, сидіння/стоячі, статус
- Форма додавання ТЗ: `plate_number`, `model`, `total_seats`, `total_standing`
- Форма редагування існуючого ТЗ
- Попередження при деактивації ТЗ з майбутніми рейсами

Функціональність:

- UC-A5.1: Власник заповнює форму та Core API створює `VEHICLES (is_active = true)`
- UC-A5.2: ТЗ стає доступним для призначення на рейси
- UC-A5.3: Редагування `total_seats` або `total_standing` → Core API оновлює лише `VEHICLES`
- UC-A5.4: **Snapshot-принцип:** зміна `VEHICLES` НЕ змінює `seats_limit_snapshot` у вже створених `TRIPS` — це поле незмінне з моменту створення рейсу

Критерії прийняття:

- Новий ТЗ одразу доступний у формі створення рейсу
- Зміна `total_seats` існуючого ТЗ не впливає на минулі та майбутні рейси
- `seats_limit_snapshot` копіюється виключно при `INSERT` рейсу

3.2 Вимоги до інтерфейсу користувача

3.2.1 Telegram Bot — Загальні вимоги до UI

ТВ-1: Навігація

- ТВ-1.1: Кнопка «← Назад» доступна на кожному екрані, крім головного меню
- ТВ-1.2: Головне меню доступне через команду `/menu` або `/start` з будь-якого стану
- ТВ-1.3: Тайм-аут inline-клавіатури — якщо пасажир не відповів протягом 10 хвилин, стара клавіатура деактивується

ТВ-2: Повідомлення про помилки

- ТВ-2.1: Усі повідомлення про помилки написані зрозумілою для користувача мовою (без технічних деталей)
- ТВ-2.2: Повідомлення про `Race Condition` містить пропозицію обрати інший рейс або спробувати знову
- ТВ-2.3: Повідомлення про блокування містить контактну інформацію диспетчера

ТВ-3: UX Водія (критичні вимоги)

- ТВ-3.1: Усі операції водія («Стоячий», «Посилка», «Підтвердити посадку», «Вирушити») виконуються максимум у 2 кліки
- ТВ-3.2: Жодна операція водія не вимагає введення тексту з клавіатури (крім опціонального коментаря до посилки)
- ТВ-3.3: Маніфест рейсу відображається одним повідомленням або пагінованим списком (по 10 пасажирів)
- ТВ-3.4: Кнопки водія великі та мінімально відрізняються від кнопок пасажирів — для запобігання помилок

ТВ-4: Відгук системи

- ТВ-4.1: Бот завжди підтверджує отримання дії (або прогрес-індикатор «🔄 Обробляємо...» для операцій >1 сек)
- ТВ-4.2: Якщо Core API недоступний, бот відображає: «Сервіс тимчасово недоступний. Зверніться до диспетчера: [номер]»

3.2.2 Web Admin Panel — Загальні вимоги до UI

WA-1: Дизайн та візуальний стиль

- WA-1.1: Інтерфейс відповідає сучасним трендам (Material Design або аналог)
- WA-1.2: Єдина кольорова схема у всій панелі
- WA-1.3: Читабельні шрифти розміром не менше 14px для основного тексту
- WA-1.4: Адекватний контраст тексту та фону (мінімум 4.5:1)

WA-2: Навігація

- WA-2.1: Бокова навігаційна панель доступна з будь-якого розділу
- WA-2.2: Кнопка «Назад» у всіх формах
- WA-2.3: Breadcrumbs для вкладених розділів

- WA-2.4: Гарячі клавіші для найчастіших дій (наприклад, Ctrl+N для нового бронювання)

WA-3: Відгук системи

- WA-3.1: Індикатори завантаження для всіх асинхронних операцій
- WA-3.2: Toast-повідомлення про успішні дії
- WA-3.3: Модальні вікна для незворотних дій (закриття рейсу, блокування пасажирів)
- WA-3.4: Валідація форм у реальному часі

WA-4: Таблиці та списки

- WA-4.1: Усі таблиці підтримують сортування по стовпцях
- WA-4.2: Усі таблиці підтримують фільтрацію та пошук
- WA-4.3: Пагінація або нескінченний скролінг для великих списків
- WA-4.4: Можливість експорту таблиці у CSV (для розділу аналітики)

3.3 Нефункціональні вимоги

3.3.1 Вимоги до продуктивності

PERF-1: Час відповіді Core API

- PERF-1.1: Пошук рейсів за датою та напрямком: ≤ 200 мс при 100 одночасних запитах
- PERF-1.2: Створення бронювання (включно з транзакційним блокуванням): ≤ 500 мс
- PERF-1.3: Зміна статусу рейсу: ≤ 300 мс
- PERF-1.4: Завантаження маніфесту рейсу (до 50 записів): ≤ 1 секунда

PERF-2: Бот

- PERF-2.1: Час реакції бота на натискання кнопки: ≤ 2 секунди (включаючи API-запит)
- PERF-2.2: Webhook-обробка подій Telegram: ≤ 3 секунди до відповіді користувачу

PERF-3: Оптимізація бази даних

- PERF-3.1: Композитний індекс (`from_location_id`, `to_location_id`, `departure_time`) у TRIPS
- PERF-3.2: Індекс (`trip_id`, `status`) у BOOKINGS для підрахунку місткості
- PERF-3.3: Індекс (`passenger_id`, `status`) у BOOKINGS для перегляду квитків пасажирів
- PERF-3.4: Індекс (`phone`) та (`telegram_id`) у USERS для швидкого пошуку

PERF-4: Масштабованість

- PERF-4.1: Система повинна підтримувати горизонтальне масштабування Core API
- PERF-4.2: Система повинна коректно працювати при 50 одночасних активних рейсах

3.3.2 Вимоги до безпеки

SEC-1: Аутентифікація та авторизація

- SEC-1.1: API-ендпоінти, що змінюють дані, вимагають автентифікації (JWT або аналог)
- SEC-1.2: Розмежування прав доступу на основі ролей (RBAC): перевірка `is_driver`, `is_admin`, `is_superuser`
- SEC-1.3: Термін дії access token: 15 хвилин; refresh token: 30 днів
- SEC-1.4: Обмеження спроб входу у Web-панель: 5 спроб за 15 хвилин
- SEC-1.5: Автоматичне завершення сесії Web-панелі після 8 годин неактивності

SEC-2: Захист даних

- SEC-2.1: Паролі адміністраторів хешуються за допомогою bcrypt або argon2 (ніколи plaintext)
- SEC-2.2: Використання HTTPS для всієї передачі даних між компонентами
- SEC-2.3: Валідація та санітизація всіх вхідних даних на рівні API
- SEC-2.4: Захист від SQL-ін'єкцій через параметризовані запити SQLAlchemy (ніколи raw SQL з інтерполяцією)
- SEC-2.5: Номери телефонів пасажирів не передаються заблокованим акаунтам

SEC-3: Аудит та логування

- SEC-3.1: Кожна зміна статусу рейсу логується з ID ініціатора та timestamp
- SEC-3.2: Кожне створення та скасування бронювання логується з `created_by_id` та `source`
- SEC-3.3: Спроби несанкціонованого доступу логуються та генерують сповіщення
- SEC-3.4: Доступ до фінансових даних та audit trail — виключно для `is_superuser = true`

3.3.3 Вимоги до надійності

REL-1: Доступність

- REL-1.1: Uptime системи: не менше 99% (максимум 7.2 години простою на місяць)
- REL-1.2: Моніторинг доступності всіх компонентів (Core API, Bot, PostgreSQL)
- REL-1.3: Автоматичне сповіщення адміністратора при виявленні збою

REL-2: Відмовостійкість

- REL-2.1: Якщо Core API недоступний — бот відображає зрозуміле повідомлення та контакт диспетчера
- REL-2.2: Транзакції PostgreSQL гарантують атомарність: часткове бронювання неможливе

- REL-2.3: Retry logic для Telegram webhook-відповідей при тимчасових збоях

REL-3: Резервне копіювання

- REL-3.1: Повне резервне копіювання БД щоденно
- REL-3.2: Зберігання резервних копій протягом 14 днів
- REL-3.3: Час відновлення системи (RTO): < 4 години
- REL-3.4: Точка відновлення (RPO): < 24 години

REL-4: Обробка помилок

- REL-4.1: Логування всіх помилок з контекстом (trip_id, user_id, action)
 - REL-4.2: Користувацькі повідомлення без технічних деталей (stack traces недоступні кінцевому користувачу)
 - REL-4.3: Circuit breaker для зовнішніх залежностей (Telegram API)
-

3.3.4 Вимоги до технологічного стеку

TECH-1: Обов'язковий стек

- TECH-1.1: Backend: Python 3.11+, FastAPI
- TECH-1.2: База даних: PostgreSQL 15+
- TECH-1.3: ORM: SQLAlchemy 2.0 з асинхронним драйвером asyncpg
- TECH-1.4: Telegram Bot: aiogram 3.x (виключно async-підхід)
- TECH-1.5: Міграції схеми БД: Alembic

TECH-2: Архітектурні обмеження

- TECH-2.1: Вся бізнес-логіка знаходиться виключно у Core API — не у боті та не у Web-панелі
 - TECH-2.2: Bot та Web-панель взаємодіють із БД виключно через Core API (не напряму)
 - TECH-2.3: Синхронний доступ до PostgreSQL (через psycopg2 тощо) заборонений
-

3.3.5 Вимоги до підтримки та обслуговування

MAINT-1: Моніторинг

- MAINT-1.1: Моніторинг доступності всіх сервісів 24/7
- MAINT-1.2: Моніторинг використання ресурсів сервера (CPU, RAM, Disk)
- MAINT-1.3: Моніторинг продуктивності БД (час запитів, кількість з'єднань)
- MAINT-1.4: Dashboard з ключовими метриками для власника

MAINT-2: Логування

- MAINT-2.1: Централізоване логування всіх компонентів
- MAINT-2.2: Рівні логування: DEBUG, INFO, WARNING, ERROR, CRITICAL
- MAINT-2.3: Структуроване логування у форматі JSON
- MAINT-2.4: Ротація логів (зберігання максимум 30 днів)

MAINT-3: Оновлення

- MAINT-3.1: Міграції БД через Alembic з контролем версій — жодних ручних змін схеми
- MAINT-3.2: Можливість rollback до попередньої версії при критичних помилках
- MAINT-3.3: Тестування оновлень на staging-середовищі перед production

MAINT-4: Документація

- MAINT-4.1: API-документація у форматі OpenAPI/Swagger (автоматично генерується FastAPI)
 - MAINT-4.2: Інструкції з розгортання та налаштування оточення
 - MAINT-4.3: Troubleshooting guide для типових проблем
-

3.3.6 Вимоги до тестування

TEST-1: Типи тестування

- TEST-1.1: Unit-тестування бізнес-логіки Core API (покриття мінімум 70%)
- TEST-1.2: Інтеграційне тестування всіх API-ендпоінтів
- TEST-1.3: End-to-end тестування критичних user flows (реєстрація → пошук → бронювання → скасування)
- TEST-1.4: Тестування Race Condition: 50 одночасних запитів на останнє місце
- TEST-1.5: Навантажувальне тестування: 100 одночасних користувачів

TEST-2: Автоматизація

- TEST-2.1: Автоматичний запуск тестів при кожному commit (CI pipeline)
- TEST-2.2: Автоматичне регресійне тестування перед деплоєм
- TEST-2.3: Тест на Snapshot-принцип: зміна VEHICLES не повинна змінювати snapshot у TRIPS

TEST-3: Критерії прийняття

- TEST-3.1: Всі критичні use cases (Must Have) проходять без помилок
- TEST-3.2: Race Condition тест: при 50 одночасних запитах на 1 місце — рівно 1 успішне бронювання
- TEST-3.3: Продуктивність відповідає метрикам PERF-1
- TEST-3.4: Відсутність критичних вразливостей безпеки (SQL injection, auth bypass)
- TEST-3.5: Покриття автоматичними тестами не менше 70% бізнес-логіки

4. Додатки

4.1 Додаток А: Use Case — Повні сценарії

UC-FULL-1: Повний цикл рейсу (Happy Path)

Актори: Власник, Диспетчер, Пасажири (×N), Водій

Основний сценарій:

1. Власник додає новий мікроавтобус у розділі «Автопарк» (`total_seats = 18` , `total_standing = 5`)
2. Диспетчер генерує розклад на тиждень — Core API копіює ліміти у `seats_limit_snapshot` та `standing_limit_snapshot` для кожного рейсу
3. Пасажир А реєструється через Telegram Bot (шеринг контакту)
4. Пасажир А знаходить рейс та бронює місце → `BOOKINGS` запис зі статусом `RESERVED` , `source = BOT`
5. Пасажир В телефонує диспетчеру → диспетчер вручну вносить бронювання → `source = PHONE` , `created_by_id = dispatcher.id`
6. Водій відкриває маніфест, бачить 2 заброньованих пасажири
7. Водій натискає «Розпочати посадку» → рейс переходить у `BOARDING`
8. Пасажир А сідає → водій натискає «✓ Підтвердити» → `status = BOARDED` , `validated_by_id = driver.id`
9. Пасажир С підсідає на проміжній зупинці → водій натискає «+ Стоячий» → новий `BOOKINGS` запис (`STANDING` , `DRIVER` , `BOARDED`)
10. Водій натискає «Вирушити» → `ACTIVE` ; водій натискає «Завершити рейс» → `COMPLETED`
11. Пасажир В не з'явився → при закритті рейсу його бронювання → `NOSHOW` ; `USER_STATS.total_noshows++`
12. Диспетчер звіряє касу з водієм → натискає «Закрити рейс» → `CLOSED`

UC-FULL-2: Race Condition — захист від подвійного продажу

Сценарій: Залишилось 1 місце. Пасажири X та Y одночасно натискають «Забронювати».

1. Обидва запити надходять до Core API майже одночасно (різниця < 100 мс)
2. Транзакція X першою отримує `SELECT ... FOR UPDATE` — встановлює блокування рядка `TRIPS`
3. Транзакція Y очікує зняття блокування
4. Транзакція X перевіряє `seats_available = 1` → створює бронювання → `commit` → блокування знімається
5. Транзакція Y отримує блокування → перевіряє `seats_available = 0` → `rollback`
6. Пасажир X отримує підтвердження бронювання
7. Пасажир Y отримує: «На жаль, останнє місце щойно зайняли. Спробуйте інший рейс»

Результат: Рівно одне бронювання створено; `overbooking` виключено.

4.2 Додаток В: Схема бази даних

```

USERS
├─ id                INT PK
├─ telegram_id       BIGINT  NULLABLE, UNIQUE  -- NULL для тінювих профілів
├─ phone             STRING  UNIQUE, INDEX
├─ full_name         STRING
├─ password_hash     STRING  -- для Web-панелі
├─ is_active         BOOL    DEFAULT true
├─ is_driver         BOOL    DEFAULT false
├─ is_admin          BOOL    DEFAULT false
├─ is_superuser      BOOL    DEFAULT false
└─ created_at        DATETIME

USER_STATS (1:1 з USERS)
├─ user_id           INT PK, FK → USERS
├─ total_trips       INT      DEFAULT 0
├─ total_noshows     INT      DEFAULT 0
└─ trust_score_cached INT      DEFAULT 100

VEHICLES
├─ id                INT PK
├─ plate_number      STRING  UNIQUE
├─ model             STRING
├─ total_seats       INT
├─ total_standing    INT
└─ is_active         BOOL    DEFAULT true

LOCATIONS
├─ id                INT PK
└─ name              STRING  UNIQUE, INDEX

TRIPS
├─ id                INT PK
├─ driver_id         INT FK → USERS
├─ vehicle_id        INT FK → VEHICLES
├─ from_location_id  INT FK → LOCATIONS
├─ to_location_id    INT FK → LOCATIONS
├─ departure_time    DATETIME INDEX
├─ arrival_time      DATETIME  NULLABLE
├─ status             ENUM('SCHEDULED', 'BOARDING', 'ACTIVE', 'COMPLETED', 'CLOSED', 'CANCELLED')
├─ seats_limit_snapshot INT  -- * НЕЗМІННИЙ snapshot з VEHICLES.total_seats
├─ standing_limit_snapshot INT -- * НЕЗМІННИЙ snapshot з VEHICLES.total_standing
├─ price_seated      DECIMAL
└─ price_standing    DECIMAL

BOOKINGS
├─ id                INT PK
├─ trip_id           INT FK → TRIPS  INDEX(trip_id, status)
├─ passenger_id      INT FK → USERS  NULLABLE  INDEX
├─ created_by_id     INT FK → USERS  NOT NULL   -- * обов'язкове
├─ created_at        DATETIME
├─ validated_by_id   INT FK → USERS  NULLABLE
├─ validated_at      DATETIME        NULLABLE
├─ booking_type      ENUM('SEATED', 'STANDING', 'PARCEL')
├─ source            ENUM('BOT', 'WEB', 'PHONE', 'INSTAGRAM', 'DRIVER') NOT NULL
├─ status            ENUM('RESERVED', 'PAID', 'BOARDED', 'CANCELLED', 'NOSHOW')
├─ passengers_count  INT    -- 0 для PARCEL
├─ amount_paid       DECIMAL
└─ comment           STRING  NULLABLE

```

4.3 Додаток С: MoSCoW пріоритетизація

Категорія

Функціональність

Ризик	Нестача часу на тестування Race Condition	Середня ймовірність	Вплив	Критичний	Планувати навантажувальне тестування з першого спринту
	Збій PostgreSQL під час пікових годин	Низька		Критичний	Реплікація + автоматичний failover

Організаційні ризики

Ризик	Ймовірність	Вплив	Мітигація
Зміна вимог у процесі розробки	Висока	Середній	Agile-методологія, короткі спринти, регулярний review цього SRS
Недооцінка складності Snapshot-логіки	Середня	Середній	Архітектурний review перед початком розробки
Нестача тестувальників	Низька	Середній	Автоматизація тестів, залучення розробників до написання тестів

Бізнес-ризики

Ризик	Ймовірність	Вплив	Мітигація
Водії не приймають нову систему	Середня	Високий	UX-тестування з реальними водіями до релізу; онбординг-сесія
Пасажири не реєструються в боті	Середня	Середній	Диспетчер зберігає можливість ручного бронювання (PHONE)
Конкуренція з існуючими рішеннями	Низька	Середній	Унікальна фіча: Driver-канал + тіньові профілі без app install

4.5 Додаток Е: Глосарій

Термін	Визначення
Overoverbooking	Ситуація, де кількість підтверджених бронювань перевищує фізичну місткість транспортного засобу
Race Condition	Стан системи, при якому два або більше конкурентних запити одночасно претендують на одне місце; усувається через <code>SELECT ... FOR UPDATE</code>
Snapshot	Незмінна копія значення конфігурації авто (seats/standing limit), збережена у таблиці TRIPS у момент створення рейсу
Тіньовий профіль	Запис у <code>USERS</code> із <code>telegram_id = NULL</code> , створений диспетчером для клієнта, що телефонує і не має Telegram
Trust Score	Кешований рейтинг надійності пасажирів (0–100): $100 - (no_shows / \max(trips, 1)) \times 100$
No-Show	Пасажир, що мав активне бронювання, але не з'явився на посадку до відправлення рейсу
Boarding Validation	Підтвердження водієм факту фізичної посадки пасажирів: заповнення <code>validated_by_id</code> та <code>validated_at</code> у <code>BOOKINGS</code>
Маніфест рейсу	Список усіх активних бронювань конкретного рейсу, що відображається водієві для управління посадкою
Audit Trail	Повний журнал дій через обов'язкові поля <code>created_by_id</code> та <code>validated_by_id</code> у таблиці <code>BOOKINGS</code>
CLOSED	Фінальний незворотний статус рейсу після фінансової звітності диспетчера з водієм
Source	Канал, через який надійшло бронювання: <code>BOT</code> / <code>WEB</code> / <code>PHONE</code> / <code>INSTAGRAM</code> / <code>DRIVER</code>
Тіньовий профіль	Запис <code>USERS</code> без <code>telegram_id</code> , що автоматично об'єднується з реальним профілем при реєстрації

Підпис та затвердження

Цей документ специфікації вимог до програмного забезпечення (SRS) для системи «Експрес Львів–Дрогобич» версії 1.0 підготовлений відповідно до стандартів IEEE 830-1998 та ISO/IEC/IEEE 29148:2018.

Підготовлено:

Команда аналітиків

Дата: 27 лютого 2026

Погоджено:

Керівник проекту: _____

Затверджено:

Головний архітектор: _____

Представник замовника: _____